

Production Web Application Outage

Interview Preparation Guide

Ticket: 002

Focus: Investigation, Analysis, Containment and Recovery

Document Date: July 30, 2026

What was your first hypothesis?

My first hypothesis was that the user-facing application path had experienced an availability failure somewhere between the client and the backend service.

I did not assume that Nginx, networking, DNS, Docker, Kubernetes, or the application was responsible. The initial ticket only established that users could not reach the application.

Why did you start there?

I started with the complete request path because a browser failure can be caused by several different layers:

```
Client → Network → TCP port 80 → Nginx → Backend application
```

Beginning broadly allowed me to test each layer independently and avoid changing systems that were still healthy.

My first checks were:

1. Confirm whether the server was reachable.
2. Test whether TCP port 80 was available.
3. Check Nginx service status.
4. Inspect listening ports.
5. Test the backend directly.

What evidence caused you to change direction?

The server remained reachable and SSH continued working, but TCP port 80 was unavailable.

Nginx was inactive, while the Flask backend on `127.0.0.1:5050` continued returning HTTP 200.

That evidence changed the investigation from a broad application or network outage to determining why Nginx had stopped.

The Nginx journal then showed an orderly shutdown instead of a crash. Authentication and sudo records showed that `webadmin` executed:

```
/usr/bin/systemctl stop nginx
```

That evidence shifted the investigation toward privileged account activity.

What evidence ruled out other possibilities?

Server failure was ruled out because:

- The host remained reachable.
- SSH remained available.
- Other services continued running.
- The operating system did not crash.

Network failure was ruled out because:

- The host responded to network testing.
- TCP port 22 remained reachable.
- Only the user-facing web port was unavailable.

DNS failure was ruled out because:

- Testing was performed directly by IP address.
- Local loopback testing bypassed DNS completely.

Backend failure was ruled out because:

- `company-web.service` remained active.
- Direct testing of port 5050 returned HTTP 200.

Nginx configuration failure was ruled out because:

- `nginx -t` passed.
- The same configuration worked after the service restarted.
- No configuration change was required.

Nginx crash was ruled out because:

- Systemd recorded an orderly stop request.
- No unexpected termination or crash evidence was present.

Docker and Kubernetes were ruled out because:

- The application path used host-based systemd services.
- Docker workloads were unrelated.
- No Kubernetes dependency existed for the application.

What monitoring would have detected this sooner?

Several controls would have detected the incident sooner:

- External HTTP health checks against the full user-facing path
- TCP port 80 availability monitoring
- Alerts when Nginx becomes inactive
- Alerts when the application stops returning HTTP 200
- SIEM detection for privileged service-control commands
- Alerts for unusual privileged SSH logins
- Alerts for repeated or failed sudo authentication
- Centralized monitoring of systemd service-state changes

The most valuable control would have been an external HTTP check because it would have detected the same failure users experienced.

If this happened in production, what would you do differently?

In production, I would use a formal incident-response process.

I would:

1. Open or update the incident ticket and establish severity.
2. Preserve volatile evidence before making unnecessary changes.
3. Notify the incident commander, application owner, security team, and business stakeholders.
4. Confirm the scope across all affected users and systems.
5. Review centralized identity, VPN, bastion, endpoint, firewall, and SIEM logs.
6. Disable or suspend the involved account through the identity provider.
7. Revoke active sessions, passwords, SSH keys, tokens, and certificates.
8. Isolate and investigate the source endpoint if credential misuse was suspected.
9. Restore service using an approved change or emergency-change procedure.
10. Validate recovery from multiple external locations.
11. Increase monitoring for repeated activity.
12. Preserve evidence according to chain-of-custody requirements.
13. Conduct a privileged-access review.
14. Complete a post-incident review and assign corrective actions.

I would not immediately rebuild the server unless evidence suggested persistence, malware, unauthorized modification, or loss of system integrity.

How did you confirm the root cause?

I correlated multiple independent sources:

- SSH authentication records
- The remote source IP address
- Failed and successful sudo activity
- The exact privileged command
- The Nginx shutdown timestamp
- Service status
- Listening-port evidence
- Backend HTTP testing
- Successful recovery after restarting Nginx

The command timestamp matched the service shutdown, and restoring Nginx restored the application without any backend, network, DNS, Docker, Kubernetes, or configuration changes.

Was the application compromised?

The evidence did not demonstrate compromise of the Flask application.

The investigation found no evidence of:

- Application-file modification
- Malicious code inside the backend
- Data theft
- Data alteration
- Persistence
- Backend process replacement

The event involved security-relevant privileged activity that caused an availability outage, but that is not the same as proving the application itself was compromised.

Was the activity malicious?

The technical evidence proves that the `webadmin` account executed the command that stopped Nginx.

It does not prove whether the action was:

- Accidental
- Intentional

- Performed by the legitimate owner
- Performed by someone using stolen or shared credentials

I would describe the activity as unauthorized or inappropriate unless an approved change record established that it was permitted.

Why did you lock the account before restoring service?

The account had an active session and had already performed an action that disrupted service.

Restoring Nginx without containment could have allowed the same session to stop it again or perform additional privileged actions.

Locking the account and terminating the session reduced immediate risk before service restoration.

How did you confirm that recovery was complete?

I validated recovery at several layers:

- Nginx reported active.
- The backend service reported active.
- TCP port 80 was listening.
- The application returned HTTP 200 through Nginx.
- Windows confirmed remote TCP connectivity.
- Browser testing succeeded.
- Both services started after reboot.
- The `webadmin` account remained locked after reboot.

This confirmed technical restoration, external reachability, user-facing functionality, and persistence across restart.

What was the most important lesson?

The most important lesson was to test each application layer independently and follow the evidence.

The backend was healthy, the server was online, and the network was functioning. Users still experienced a complete outage because the reverse proxy was stopped.

A disciplined investigation prevented incorrect conclusions and unnecessary changes to healthy components.